

SUSHI MIYU

Proyecto de Fin de Grado



Adriana Salazar Daza
Joaquín Ruíz Jiménez
Ainhoa Quintero González
2º Daw tarde

Curso 2025/2026

SUSHI MIYU:

ÍNDICE:

1.INTRODUCCIÓN:	
1.1 CONTEXTO DEL PROYECTO:	5
1.2 JUSTIFICACIÓN DEL PROYECTO:	5
1.3 OBJETIVOS DEL PROYECTO:	6
2. CONTEXTO EMPRESARIAL:	
2.1 IDEA DE NEGOCIO:	7
2.2 IDENTIDAD DE LA MARCA:	7
DISEÑO DEL LOGO :	7
2.3 PÚBLICO OBJETIVO:	8
2.4 PROPUESTA DE VALOR:	8
3. ANÁLISIS DE REQUISITOS:	
3.1 REQUISITOS FUNCIONALES:	9
3.2 REQUISITOS NO FUNCIONALES:	10
4. DISEÑO DE LA SOLUCIÓN:	
4.1 ARQUITECTURA GENERAL:	11
4.2 JUSTIFICACIÓN TECNOLÓGICA:	11
4.3 DISEÑO VISUAL Y EXPERIENCIA DE USUARIO:	12
5. DESARROLLO DEL FRONTEND:	
5.1 FRONTEND PÚBLICO – ANGULAR:	13
5.1.1 Objetivos:	13
5.1.2 Arquitectura:	13
5.1.3 Rutas y navegación:	14
5.1.4 Componentes de la interfaz:	14
5.1.5 Consumo de API REST:	14
5.1.6 Autenticación y seguridad:	14
5.1.7 Manejo de errores:	15
5.1.8 Diseño responsive:	15
5.1.9 Testing y automatización:	15
5.1 DASHBOARD ADMINISTRATIVO – REACT:	15
5.2.1 Objetivos:	15
5.2.2 Arquitectura:	15
5.2.3 Navegación:	16
5.2.4 Gestión de productos y categorías:	16
5.2.3 Gestión de usuario:	16
5.2.4 Notificaciones (Toast):	16

5.2.5 Diseño visual:.....	16
5.2.6 Seguridad:.....	16
6. DESARROLLO DEL BACKEND:.....	
6.1 OBJETIVOS:.....	17
6.2 ARQUITECTURA:.....	17
6.3 ENDPOINTS:.....	17
6.4 SEGURIDAD Y AUTENTIFICACIÓN:.....	18
6.4.1 Sanctum:.....	18
6.4.2 Gestión de roles:.....	18
6.4.3 Role middleware:.....	18
6.4.4 Validación de datos:.....	18
6.5 BASE DE DATOS:.....	19
6.5.1 Modelo entidad-relación:.....	19
6.6 GESTIÓN DE IMÁGENES:.....	20
6.7 BLADES:.....	20
6.8 DOCUMENTACIÓN SWAGGER:.....	20
7. DESPLIEGUE:.....	
7.2 EC2:.....	23
7.3 APPSPEC.YML:.....	24
7.4 GITHUB ACTIONS Y CODE DEPLOY:.....	24
7.5 TERRAFORM:.....	25
7.6 FLUJO DE EJECUCIÓN:.....	25
8. METODOLOGÍA DE TRABAJO:.....	
8.1 ORGANIZACIÓN:.....	26
8.2 TRELLO Y SPRINTS:.....	26
8.3 CONTROL DE VERSIONES:.....	26
8.3.1 Estructura de ramas:.....	26
8.3.2 Convención de commits:.....	27
8.3.3 Release y versionado:.....	27
9. CONCLUSIONES:.....	
9.1 OBJETIVOS ALCANZADOS:.....	28
9.2 VALORACIÓN TÉCNICA:.....	28
9.3 APRENDIZAJES OBTENIDOS:.....	28
9.4 MEJORAS PROPUESTAS:.....	29

RESUMEN:

El presente trabajo tiene como objetivo el desarrollo al completo de una página web integral para un restaurante ficticio denominado Sushi Miyu, planteado dentro de un contexto empresarial realista donde una pequeña empresa que próximamente abrirá sus puertas por primera vez necesita una solución tecnológica completa para la gestión de su negocio. Este desarrollo cubrirá todo el proceso al completo, desde el diseño, desarrollo y despliegue de la aplicación web.

El sistema cubre tanto las necesidades de los clientes como las del personal interno del restaurante mediante una arquitectura dividida en una aplicación pública desarrollada en Angular destinada a los clientes y un panel administrativo construido en React para la gestión interna del restaurante. También para la parte del backend se desarrollará una Api Rest implementada en Laravel que centralizará la lógica de negocio y el acceso a los datos.

Entre las funciones implementadas se destaca la visualización del catálogo de productos clasificado por categorías, registro e inicio de sesión de usuarios, la gestión de roles y permisos y la creación y administración de pedidos.

A nivel técnico el proyecto hace uso de tecnologías ampliamente utilizadas en entornos profesionales como Angular, React, Laravel 12 y Laravel Sanctum, MySQL, Docker, AWS y Terraform. Asimismo el proyecto contiene elementos de un entorno empresarial real incluyendo el diseño de identidad visual de marca, despliegue en infraestructura cloud, control de versiones mediante Git y GitHub, CI/CD usando GitHub Actions, documentación de API mediante Swagger y organización del desarrollo basada en trabajo colaborativo.

El resultado es una plataforma escalable, modular y preparada para responder a necesidades operativas internas y a la experiencia de los clientes.

1 - INTRODUCCIÓN:

1.1 CONTEXTO DEL PROYECTO:

La digitalización es un elemento imprescindible para cualquier negocio, y para la restauración no lo es menos. La administración centrada en productos y clientes y la gestión de pedidos entre otras cosas son aspectos fundamentales para ofrecer un servicio eficiente y competitivo.

Dentro de este contexto surge Sushi Miyu, una propuesta empresarial ficticia basada en un restaurante de comida asiática, próxima a comenzar su actividad comercial y requiere de una plataforma tecnológica capaz de cubrir tanto la parte pública orientada al cliente como la gestión del negocio.

Para responder a su necesidad, se plantea el desarrollo de una solución compuesta por una aplicación web pública, un panel administrativo y una API REST centralizada, todo ello desarrollado por un pequeño equipo de tres desarrolladores que actúan como empresa contratada para la entrega del producto antes de la apertura del restaurante.

El proyecto se ha desarrollado siguiendo criterios de un entorno profesional real, incluyendo control de versiones, organización modular y separación de responsabilidades entre sistemas, documentación técnica y despliegue en infraestructura cloud.

1.2 JUSTIFICACIÓN DEL PROYECTO:

El sector de la restauración requiere cada vez más soluciones digitales capaces de ofrecer una experiencia cómoda y eficiente tanto para el usuario final como para los trabajadores.

Durante el análisis inicial de necesidades se identificaron diversos problemas que se debían resolver:

- Ausencia de una plataforma digital para mostrar el menú al público.
- Necesidad de centralizar la gestión de productos y categorías.
- Falta de un sistema de autenticación y gestión de usuarios.
- Necesidad de automatizar el proceso de pedidos.
- Requerimiento de una plataforma administrativa sencilla para el personal
- Necesidad de una infraestructura escalable y desplegada en la nube.

Además, el proyecto permite aplicar los conocimientos adquiridos en diferentes módulos del ciclo de Desarrollo de Aplicaciones Web incluyendo desarrollo frontend, backend, bases de datos, despliegue cloud, seguridad, experiencia de usuario y gestión empresarial.

1.3 OBJETIVOS DEL PROYECTO:

OBJETIVOS GENERALES:

Diseñar, desarrollar y desplegar una aplicación web integral para un restaurante de sushi que permita gestionar productos, usuarios y pedidos, ofreciendo una experiencia digital que pueda ser accesible para clientes y trabajadores.

OBJETOS ESPECÍFICOS:

- Diseñar una interfaz pública moderna y responsive para visualizar el catálogo del restaurante.
- Implementar una clasificación de productos mediante categorías.
- Desarrollar un sistema de autenticación y registro de usuarios.
- Implementar protección de rutas y control de acceso mediante roles.
- Crear una API REST robusta y documentada.
- Diseñar un dashboard administrativo para la gestión interna intuitiva.
- Gestionar la información y datos del restaurante mediante una base de datos relacional.
- Implementar una infraestructura cloud utilizando AWS.
- Automatizar el despliegue mediante Terraform.
- Aplicar buenas prácticas de desarrollo, versionado y trabajo colaborativo.

2 - CONTEXTO EMPRESARIAL:

2.1 IDEA DE NEGOCIO:

Sushi Miyu nace como un restaurante especializado en cocina asiática moderna orientada tanto al consumo en el local como al servicio para llevar.

La idea empresarial se basa en ofrecer una experiencia gastronómica accesible, moderna y digitalizada donde los clientes puedan consultar el menú online, registrarse, realizar pedidos y mantenerse conectados a la marca desde cualquier dispositivo.

La empresa busca posicionarse con una imagen moderna capaz de atraer especialmente a un público joven y familiar y con una identidad visual sólida.

2.2 IDENTIDAD DE LA MARCA:

Uno de nuestros elementos fundamentales para el proyecto era definir el diseño de la identidad corporativa del restaurante, desde buscar un nombre, eslogan y logo de la marca.

El nombre de la empresa “Sushi Miyu” proviene de las palabras en chino 谜语 donde Miyu significa “acertijo”. Este término fue elegido por su sencillez al pronunciarlo y al escribirlo, además de ser una palabra corta fácil de recordar. Estos aspectos son claves para poder elegir un nombre que los clientes puedan recordar y distinguir de otros restaurantes.

El branding fue diseñado para mantener una estética minimalista, elegante y fácilmente reconocible, además de poder conectar el producto y la cultura asiática con la marca.

DISEÑO DEL LOGO :

Para el diseño del logo se buscaba transmitir:

- Tradición gastronómica oriental.
- Modernidad y minimalismo.
- Identidad visual memorable.
- Coherencia estética con la experiencia del restaurante.

Para ello el logo fue diseñado inspirándose en la escritura asiática y en formas visuales relacionadas con el sushi y la gastronomía oriental. Además de que fuera fácilmente reconocible, se buscaba también que fuera adaptable a formatos digitales en diversos tamaños y sobre distintos fondos y temas.



ESLOGAN CORPORATIVO:

Para reforzar la identidad de marca se planteó un eslogan con los valores del restaurante. Busca transmitir la idea de que el restaurante no se limita únicamente a ofrecer comida, sino una experiencia completa basada en sabor, diseño y cultura.

“Más que sushi, una experiencia Miyu”

PALETA DE COLORES CORPORATIVA:

La identidad visual se construyó a base de una paleta de color monocromática basada en tres colores principales:

- Rojo: representa energía, pasión y gastronomía. Además está asociado con la cultura asiática.
- Negro: elegancia, sofisticación y minimalismo.
- Gris: ofrece neutralidad visual y modernismo.

Estos son los colores principales que se pueden observar durante todo el proyecto para mantener la coherencia estética.

2.3 PÚBLICO OBJETIVO:

Sushi Miyu se orienta principalmente hacia estos perfiles de usuarios:

- Jóvenes adultos (18–35 años): personas habitadas al consumo de comida asiática y a la comodidad de los pedidos online.
- Familias: interesadas en experiencias gastronómicas accesibles y modernas.
- Clientes de take away: personas que prefieren disfrutar en la comodidad de su hogar.
- Influencers o foodies: jóvenes que quieran mostrar la estética y experiencia de nuevos locales abiertos con productos fuera de lo común.

2.4 PROPUESTA DE VALOR:

La propuesta de valor de Sushi Miyu se basa en combinar una experiencia gastronómica de inspiración japonesa con una fuerte digitalización del servicio.

Mientras muchos pequeños restaurantes dependen de procesos manuales o plataformas externas, Sushi Miyu busca centralizar toda su operativa dentro de una solución propia y personalizada.



3 - ANÁLISIS DE REQUISITOS:

Tras haber recopilado la información necesaria y puntos que el cliente busca, se definieron una serie de requisitos funcionales y no funcionales para poder realizar el desarrollo de una forma más organizada.

3.1 REQUISITOS FUNCIONALES:

01 – Visualización del catálogo de productos: El usuario podrá navegar por el menú del restaurante y visualizar información de los productos incluyendo nombre, descripción, precio e imagen.

02 – Clasificación por categorías: Los productos están organizados por categorías para facilitar la navegación y experiencia de usuario.

03 – Registro de usuarios: el usuario podrá registrarse mediante un formulario proporcionando nombre, correo electrónico y contraseña. Por defecto será creado con rol de “user”.

04 – Inicio de sesión: Los usuarios podrán utilizar las credenciales con las que se han registrado para poder autenticarse y realizar funciones dentro de la página.

05 – Protección de rutas privadas: el sistema restringe el acceso a determinadas zonas según el rol del usuario.

06 – Realización de pedidos: los usuarios podrán seleccionar productos y generar pedidos asociados a su cuenta.

07 – Recepción de pedidos en administración: El sistema centraliza los pedidos realizados por clientes para su gestión desde cocina o administración.

08 – Gestión de productos: desde el dashboard administrativo será posible realizar operaciones de creación, edición y eliminación de productos (CRUD).

09 – Gestión de categorías: El administrador podrá crear, modificar o eliminar categorías.

10 – Gestión de usuarios: el administrador será capaz de crear nuevos usuarios desde el dashboard pudiendo incluso especificar su rol, además de eliminarlos o de cambiar el rol.

11 – Gestión de roles: El sistema contará con al menos tres perfiles de usuario: administrador, trabajador y usuario. Esto permitirá dar diferentes permisos a cada uno.

12 – API REST: la lógica del negocio estará centralizada en Laravel y será consumida desde Angular y React.

13 – Documentación técnica de endpoints: La API será documentada utilizando Swagger/OpenAPI.

3.2 REQUISITOS NO FUNCIONALES:

01 – Seguridad: el sistema deberá implementar autenticación segura, para ello se usará Laravel Sanctum, hash de contraseñas, middleware de autenticación y protección de rutas.

02 – Escalabilidad: al separar frontend y backend y despliegue cloud permite añadir nuevas funcionalidades sin comprometer el sistema existente.

03 – Mantenibilidad: el código deberá seguir una estructura modular y organizada gracias al uso de componentes, servicios, controladores y MVC.

04 – Diseño responsive: el frontend fue adaptado para diversos dispositivos y resoluciones.

05 – Usabilidad: la navegación debe ser sencilla e intuitiva para el usuario con una jerarquía visual clara, diseño minimalista y una buena organización visual, siguiendo la regla de los 3 clics.

06 – Disponibilidad: el sistema debe ser accesible desde internet gracias al despliegue cloud mediante AWS.

4 - DISEÑO DE LA SOLUCIÓN:

Una vez identificadas las necesidades del cliente y los requisitos, se procede al diseño de la arquitectura general del sistema.

Para que fuera escalable y modular se optó por una arquitectura basada en varios componentes, manteniendo separadas la parte del cliente, administración y lógica de negocio.

4.1 ARQUITECTURA GENERAL:

El proyecto se encuentra dividido en tres bloques principales:

1. Aplicación pública (Angular).
2. Dashboard administrativo (React).
3. API REST y base de datos (Laravel y MySQL).

4.2 JUSTIFICACIÓN TECNOLÓGICA:

ANGULAR:

La aplicación pública fue desarrollada utilizando Angular, ya que nos permite estructurar aplicaciones de forma organizada y mantenible. Tiene un routing robusto, alta escalabilidad y una integración sencilla con APIs REST. Además utilizarlo era un requerimiento del módulo de Entorno Cliente.

REACT:

El dashboard administrativo fue desarrollado utilizando React con TypeScript. React tiene una mayor capacidad de modularización y rapidez visual. El uso de un tipado estático reduce errores y mejora la calidad del código.

LARAVEL:

El backend fue desarrollado utilizando Laravel 12, lo que nos permite desarrollar una API REST bien estructurada gracias al MVC, seguridad incorporada con sanctum, un sistema robusto de autenticación y una gran productividad y documentación.

MySQL:

El sistema gestor de bases de datos se implementa mediante MySQL, que tiene un alto rendimiento, integridad relacional y compatibilidad con Laravel.

AWS:

El despliegue se realizó utilizando Amazon Web Services que nos permite gestionar servidores remotos, crear IP Elásticas y acceder de forma segura con SSH.

TERRAFORM:

Fue seleccionado para automatizar el despliegue de la infraestructura pudiendo reducir errores manuales y facilitando la replicación del entorno.

DOCKER:

Docker fue utilizado principalmente en el backend, lo que nos permite un despliegue mucho más sencillo, más consistencia entre entornos y tener que configurar menos dependencias.

GitHub y GitHub Actions:

Github es el más usado sistema de control de versiones del mundo, es con el que más experiencia tenemos y es tremendamente simple de utilizar y entender incluso para alguien que nunca lo haya usado.

Github Actions nos permite automatizar la integración continua y el despliegue continuo de nuestra aplicación desde nuestro repositorio a las respectivas instancias creadas en AWS, haciendo uso de Github Actions Secrets y haciendo un sistema más completo, seguro y escalable.

4.3 DISEÑO VISUAL Y EXPERIENCIA DE USUARIO:

El diseño frontend fue desarrollado siguiendo los principios de experiencia de usuario y coherencia visual, tanto en la parte pública de Angular como en el panel administrativo de React.

Se utilizó la paleta de colores corporativos (rojo, negro y gris) que permiten transmitir una sensación de elegancia, modernidad y coherencia con la gastronomía asiática.

Se buscaba:

- Simplicidad visual.
- Navegación intuitiva.
- Diseño responsive.
- Minimalismo.
- Claridad jerárquica.

Se empleó Figma como herramienta de prototipado previo al desarrollo para verificar que fuera un diseño funcional y atractivo. Puedes verlo a través de este enlace: [Figma](#).

5 - DESARROLLO DEL FRONTEND:

El desarrollo del Frontend se dividió en dos aplicaciones independientes para separar la parte pública y la parte privada administrativa.

5.1 FRONTEND PÚBLICO – ANGULAR:

La parte pública de Sushi Miyu fue desarrollada utilizando Angular 20 LTS, con una arquitectura basada en componentes standalone y servicios reutilizables.

5.1.1 Objetivos:

Fue diseñada para cumplir estas funciones:

- Mostrar el catálogo del restaurante.
- Organizar productos por categorías.
- Facilitar navegación intuitiva.
- Implementar autenticación de usuarios.
- Realización de pedidos.
- Adaptarse a diferentes tamaños de pantalla.

5.1.2 Arquitectura:

Se organizó mediante una estructura modular con la finalidad de separar responsabilidades.

- **Páginas** de la aplicación.
- **Componentes** que componen la aplicación divididos en carpetas. La carpeta shared donde se encuentran los componentes que se reutilizan a lo largo de toda la aplicación, como puede ser el header o el logo. Para la comunicación entre componentes se utilizaron Inputs, Outputs y Servicios.
- **Servicios** para peticiones http a la API. Aquí es donde se encuentra toda la lógica y el mapeo de datos que llegan del backend que serán posteriormente redirigido a las páginas que requieran esa información.
- **Interfaces** para el tipado de datos. Angular utiliza por defecto TypeScript, y para ello se requieren de interfaces.
- **Environments** para tener en un solo lugar las rutas para poder conectarse al backend y a React.
- **Imágenes** utilizadas de portada y de fondo en algunas páginas. Creadas con inteligencia artificial.

5.1.3 Rutas y navegación:

La navegación entre páginas fue implementada mediante el sistema de routing de Angular. Gracias a esto podemos definir las rutas de la aplicación (home, menu, login...) incluyendo una ruta comodín (wildcard route) para páginas inexistentes.

5.1.4 Componentes de la interfaz:

Una de las bases del desarrollo frontend es el uso de componentes reutilizables que requieren de renderizar en la mayoría de las páginas. Un ejemplo de esto son los componentes que componen la interfaz de la página:

- El **header** es el componente que aparece arriba del todo una vez que alguien entra a la página. Permite navegar al carrito y desplegar el menú lateral (SideNavbar) mediante un servicio.
- El **menú lateral** permite la navegación entre páginas.

Otros componentes también utilizados son el Product Card para mostrar visualmente cada producto del restaurante y el Category Section para agrupar productos de una categoría concreta.

5.1.5 Consumo de API REST:

La aplicación se comunica con el backend de Laravel mediante peticiones HTTP, devolviendo respuestas en formato JSON que son posteriormente tipadas gracias a las interfaces en TypeScript.

Un ejemplo de cómo funciona el flujo sería el siguiente:

1. Usuario entra a /menu.
2. Se carga el MenuPage.
3. Se llama al ProductService.
4. Realiza la petición GET de /api/products.
5. Llega a la API de Laravel.
6. Busca los datos en la base de datos de MySQL.
7. Devuelve una respuesta JSON que será tipada en el servicio.
8. El servicio envía la lista de los productos a la página del menú para que se muestre.

5.1.6 Autenticación y seguridad:

Implementa autenticación basada en tokens para mantener sesiones seguras y proteger funcionalidades privadas. Tras el login:

1. El usuario introduce credenciales.
2. Laravel valida el usuario.
3. Se devuelve un token Sanctum.

4. El token se almacena en el navegador mediante el LocalStorage.

5.1.7 Manejo de errores:

El control de errores se implementó utilizando bloques try catch y control de respuestas HTTP mostrando un feedback visual al usuario. Se contemplan errores como:

- Login invalido.
- Usuario inexistente.
- Error de conexión.
- Error del servidor.

5.1.8 Diseño responsive:

Uno de los objetivos fue garantizar la adaptabilidad en diversos dispositivos. Para ello se utilizó flexbox, Grids y Scss.

5.1.9 Tests de Angular:

Para asegurar el correcto funcionamiento del código, hemos hecho uso de Karma, una herramienta de ejecución de tests para Angular que comprueba diferentes partes del código, fácil de usar, segura y con pruebas en entornos de ejecución reales (Karma hace sus pruebas en el navegador, en concreto nosotros hemos usado Chrome Headless para que no influya en la experiencia de usuario).

5.1 DASHBOARD ADMINISTRATIVO – REACT:

La parte privada del frontend fue desarrollada utilizando React 19 con TypeScript, permitiendo gestionar productos, usuarios, categorías y pedidos.

5.2.1 Objetivos:

Fue diseñado para:

- Gestionar productos.
- Gestionar categorías.
- Administrar usuarios.
- Supervisar pedidos.
- Restringir accesos según rol.
- Simplificar operaciones internas.

5.2.2 Arquitectura:

De forma similar a Angular el proyecto se estructuró:

- **Páginas** de la aplicación.
- **Componentes** de cada página y de la interfaz.
- **Servicios** para agrupar la lógica de las peticiones a la API.

- **Interfaces** (types) necesarias ya que se utiliza TypeScript.
- **Hook** para obtener el rol del usuario.
- **Environments** para almacenar las rutas para la conexión con el backend.

5.2.3 Navegación:

La navegación se implementó mediante React Router DOM. Las rutas principalmente son para poder ver las paginas, las paginas del CRUD.

5.2.4 Gestión de productos y categorías:

Aquellos usuarios con rol de trabajador únicamente podrán ver la lista de productos y categorías, en cambio si eres administrador podrás ver los botones al lado que te permitirá poder eliminar y editar, además de poder crear nuevos. Para una mayor facilidad encontrando los productos, estos se pueden filtrar por categoría y por nombre.

5.2.3 Gestión de usuario:

Únicamente los administradores serán capaces de visualizar y administrar los usuarios registrados. Es posible eliminar usuarios, crear nuevos pudiendo elegir desde un principio su rol y cambiar el rol de usuarios existentes.

5.2.4 Notificaciones (Toast):

Se implementó un sistema de notificaciones visuales que permiten informar al usuario de que la operación fue realizada con éxito o si ha habido algún error. Un ejemplo puede ser al crear un nuevo producto o modificar un usuario. Este aparecerá abajo a la derecha con un mensaje personalizado para cada acción.

Además al eliminar un usuario te aparecerá un pop up que tendrás que confirmar antes de que se elimine definitivamente el elemento, avisando por ejemplo de que al eliminar una categoría todos los productos vinculados a esta serán eliminados.

5.2.5 Diseño visual:

El dashboard sigue una línea visual oscura y minimalista. Se ha utilizado Tailwind CSS para los estilos y Lucide React para los iconos.

5.2.6 Seguridad:

El acceso al panel administrativo se encuentra restringido para aquellos usuarios que tengan una sesión iniciada con un usuario con un rol de trabajador o administrador. React comprobará este rol buscando en el LocalStorage y en caso de no encontrarlo te mostrará una pantalla con un formulario para poder iniciar sesión.

Además el CRUD de los datos almacenados en el backend únicamente serán permitidos para los administradores, al igual que solo ellos podrán ver la lista de usuarios y entrar en la página de ventas.



6 - BACKEND Y API REST:

El backend es el encargado de gestionar la lógica de negocio, acceso a los datos, la autenticación y la comunicación con la base de datos. La API REST actúa como punto de comunicación entre el Frontend (Angular y React) y la base de datos MySQL.

Para su desarrollo se utilizó Laravel 12 que es ampliamente utilizado en entornos profesionales debido a su arquitectura robusta, seguridad integrada y elevada productividad.

6.1 OBJETIVOS:

Fue diseñado para cubrir estas necesidades:

- Centralizar la lógica de negocio.
- Gestionar autenticación de usuarios.
- Administrar roles y permisos.
- Gestionar productos y categorías.
- Procesar pedidos.
- Proteger información sensible.
- Ofrecer una API reutilizable y documentada.

6.2 ARQUITECTURA:

El sistema fue desarrollado siguiendo el patrón MVC (Modelo-Vista-Controlador) propio de Laravel.

- En los **modelos** podemos definir la estructura de los datos y las relaciones entre las tablas.
- Las **vistas** que incluye Laravel son las Blades
- Los **controladores** se encargan de gestionar las peticiones HTTP, validaciones, manejo de errores y respuestas JSON.

6.3 ENDPOINTS:

Principalmente se utilizan métodos GET, POST, PUT y DELETE para los endpoints.

Los endpoints principalmente son los que corresponden al CRUD de productos, categorías y usuarios, además de aquellos utilizados para la autenticación como el login, registro y logout y algunos extras para facilitar los datos al frontend como la lista de productos clasificados por categorías.

La lista completa de endpoints se puede observar en la documentación de Swagger.

6.4 SEGURIDAD Y AUTENTIFICACIÓN:

El sistema implementa distintos mecanismos de protección con el fin de garantizar confidencialidad y control de acceso a los datos.

6.4.1 Sanctum:

La autenticación fue implementada mediante Laravel Sanctum basada en tokens de acceso. Una vez que laravel valida las credenciales se genera el token que es enviado al frontend para poder realizar las peticiones.

6.4.2 Gestión de roles:

Además se controla el acceso mediante roles de usuario. Para ello se definieron tres perfiles, siendo user el más bajo y admin el más alto:

- Admin:
 - Crear, editar y eliminar productos y categorías.
 - Ver, crear, editar y eliminar los usuarios.
- Worker:
 - Todo lo que puede realizar un usuario estándar.
 - Visualizar pedidos.
- User:
 - Registrarse.
 - Iniciar sesión.
 - Consultar menú.
 - Realizar pedidos.
 - Consultar pedidos propios.

6.4.3 Role middleware:

Para la protección de rutas se creó un middleware que permite restringir el acceso según el rol del usuario autenticado.

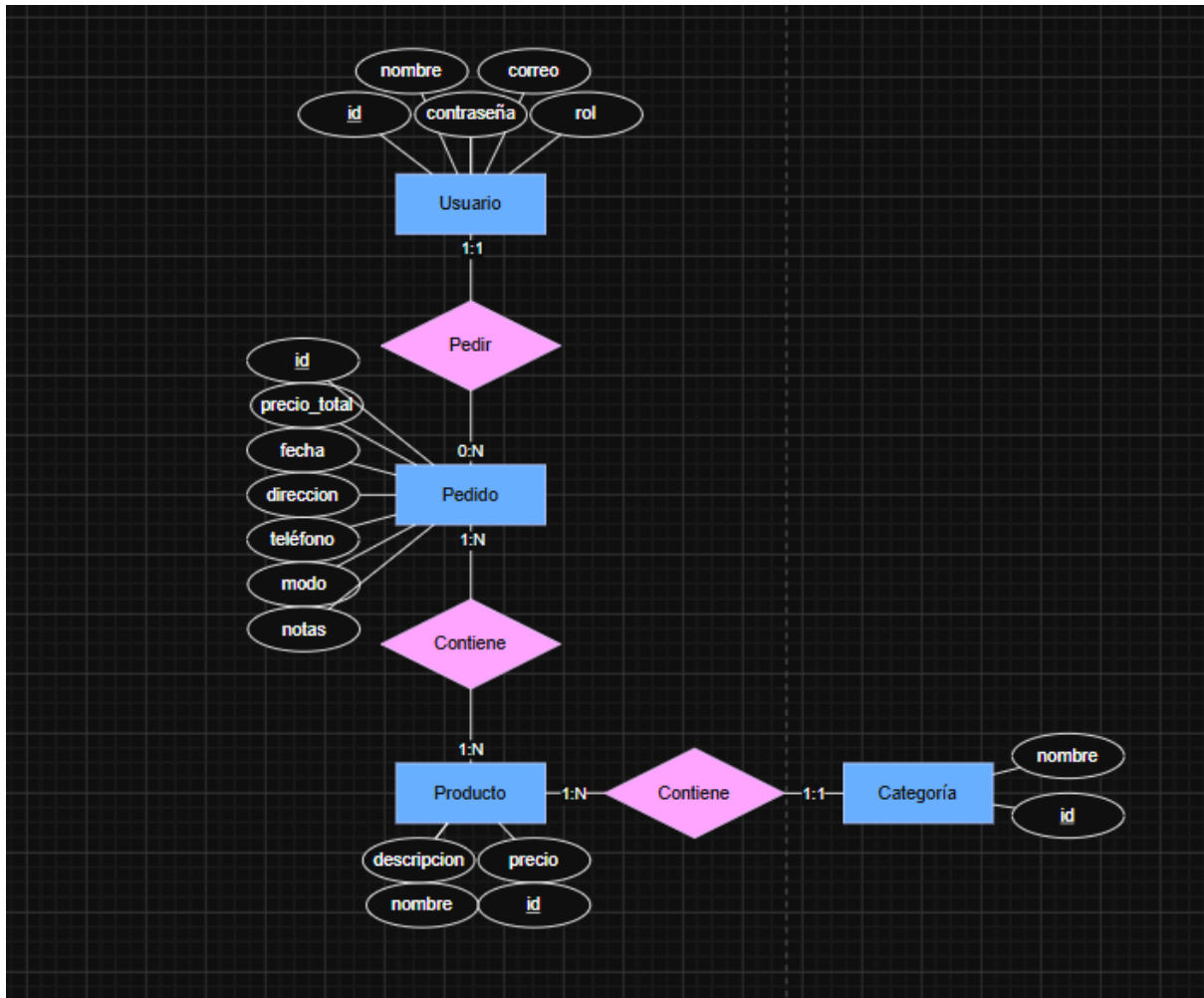
6.4.4 Validación de datos:

Laravel permite validar la entradas antes de procesarlas, de esta forma evitamos errores o inconsistencias. Ejemplo de validaciones utilizadas son una longitud mínima para la contraseña, números positivos y campos obligatorios.

Esto permite devolver respuesta de los errores HTTP como el 200 para los que han funcionado correctamente, 403 para los que no tienes permisos o 500 si es un error del servidor.

6.5 BASE DE DATOS:

La persistencia de datos fue implementada utilizando MySQL 8. Toda la estructura fue implementada mediante Migraciones, Seeders y Factories para reproducir el entorno completo automáticamente.



6.5.1 Modelo entidad-relación:

Se basa en cuatro entidades principales: usuario, producto, categoría y pedido. Sabiendo la relación entre ellos sacamos las siguientes tablas mediante migraciones:

- Users.
- Categories.
- Products.
- Orders.
- Order_product: surge de la relación de muchos a muchos entre los pedidos y los productos.
- Personal_access_tokens: se genera automáticamente para la generación de los tokens.

- Add_status_to_orders: gracias a esta migración se puede añadir estado en los pedidos (en proceso, finalizado...).

Para poder insertar datos en las tablas se utilizaron los seeders. Lo cual facilita las pruebas durante el desarrollo.

6.6 GESTIÓN DE IMÁGENES:

Se utilizó Laravel Storage para almacenar archivos, como las imágenes de los productos. Este se encuentra en “storage/app/public/products/” y se expone gracias a crear un enlace simbólico, que nos permitirá acceder a estas imágenes a través de una ruta pública.

6.7 BLADES:

Laravel nos incluye vistas Blades para poder realizar un dashboard básico, poder registrarse e iniciar sesión y gestionar los datos almacenados.

Se utilizó además **Laravel Breeze** como starter kit de autenticación, que también nos ofrece blades, rutas y funciones para el login, registro e incluso el poder cambiar la contraseña.

Están creadas las vistas para realizar el CRUD de productos y categorías, permitiendo únicamente el acceso a usuarios con perfil de administrador.

6.8 DOCUMENTACIÓN SWAGGER:

La documentación de los endpoints de la API fue documentada utilizando Swagger/OpenAPI. El objetivo fue proporcionar:

- Descripción de endpoints.
- Métodos disponibles.
- Ejemplo de peticiones y respuestas.

7 - DESPLIEGUE:

Uno de los objetivos fundamentales del proyecto consistía en desplegar la aplicación en una infraestructura cloud real que permita garantizar accesibilidad y disponibilidad del sistema. Para ello se optó por utilizar Amazon Web Services (AWS) como proveedor principal.

Para la parte de desplegar el servidor, usaremos una instancia de EC2, CodeDeploy para el correcto despliegue, para mantener la permanencia usamos un bucket de S3 y IAM para administrar permisos para acceder a las diferentes fuentes.

Las tecnologías utilizadas fueron:

- AWS EC2.
- AWS S3
- Terraform.
- Docker.
- Apache.
- GitHub.
- Pipelines CI/CD.

7.1 OBJETIVOS:

El sistema de despliegue debía cumplir los siguientes objetivos:

- Hospedar el sistema en infraestructura cloud real.
- Garantizar accesibilidad remota.
- Permitir la automatización del aprovisionamiento.
- Separar frontend y backend.
- Facilitar el mantenimiento del sistema.

Además debía cumplir con los requisitos académicos mínimos establecidos:

- Uso de AWS
- Uso de EC2
- Infraestructura declarada mediante Terraform.
- Acceso SSH.
- IP Elástica.
- DNS
- HTTPS.
- Automatización mediante pipelines.

7.2 EC2:

Sirve para crear instancias que funcionan como máquinas virtuales y nos permiten modificarlas a nuestro antojo. Se pueden crear de forma personalizada (con más o menos RAM, procesador, almacenamiento, etc) y son muy escalables.

En nuestro caso, usamos una instancia t2.medium para guardar nuestro React y Angular para el frontend y, dockerizado en el backend, Laravel, MySQL como base de datos y phpMyAdmin para la administración de la BBDD.

Como punto extra, nuestro servidor se puede enlazar a un proveedor de DNS (como DuckDNS) para permitirnos usar un nombre de dominio como [Sushimiyu.duckdns.com](https://sushimiyu.duckdns.com).

7.3 APPSPEC.YML:

Este archivo propio de AWS se encarga de dar las instrucciones al CodeDeploy sobre que archivos copiar sobre el ordenador o servidor y que scripts ejecutar antes o después del proceso de implementación.

Contiene información sobre el sistema operativo de destino, los archivos del código fuente que se ejecutarán y/o modificarán y los hooks. Estos últimos son algo así como “puntos de enlace” desde los que se pueden ejecutar scripts personalizados.

7.4 GITHUB ACTIONS Y CODE DEPLOY:

Es el encargado del despliegue continuo y de la integración continua del código en AWS, automatizando algo que anteriormente había que hacer manualmente:

- CI (Continuous Integration):
 - Al hacer merge, se actualiza el código en AWS al par de minutos.
 - Se asegura de que el código nuevo se integre sin romper el anterior.
- CD (Continuous Delivery)
 - Se encarga de realizar distintos tipos de test
 - Asegura una integración limpia tras el CI
- CD(Continuous Development)
 - Extensión del Continuous Delivery
 - Se encarga de desplegar el código de forma segura y automática
 - Login AWS, AWS cli,

De la mano con todo esto, está AWS Codedeploy.

Github Actions se va a encargar de toda la parte del CI, mientras que AWS CodeDeploy se encarga de la parte de CD.

En nuestro caso, CodeDeploy ha sido una herramienta que hemos tenido en cuenta, pero tras valorar los pros y contras, no ha parecido mejor y más interesante utilizar GitHub Actions para el CI/CD completo, ya que usar CodeDeploy significaba tener divididos el CI y el CD de forma independiente, lo que en producción podría darnos problemas, además de la complejidad de la propia instalación y uso de CodeDeploy y su daemon CodeDeploy Agent

7.5 TERRAFORM:

Terraform es una herramienta de infraestructura como servicio que permite aprovisionar o gestionar recursos tecnológicos como servidores o redes a plataformas en la nube mediante archivos de configuración legibles y versionables.

En nuestro proyecto, usaremos terraform para crear las instancias de EC2, buckets de S3, grupos de seguridad para el acceso por ssh y peticiones http, roles y permisos mediante IAM y ejecutar scripts en el momento de instalación.

Es gracias a terraform, que se sincroniza con github actions, que podemos hacer cambios en la nube sin tener que acceder manualmente, ya que todas estas partes se unen para permitir un correcto flujo de trabajo.

7.6 FLUJO DE EJECUCIÓN:

Para mostrar un flujo de trabajo normal en la aplicación, podemos ver el siguiente:

Con terraform:

- Terraform init
- Terraform apply
- Terraform crea la instancia EC2, el bucket en S3 y CodeDeploy

Al hacer cambios en una rama en github:

1. Github Actions se encarga de conectarse a la instancia mediante ssh, haciendo uso de los permisos que hemos administrado con Terraform.
2. Además realiza los test, modifica el código y junto a CodeDeploy se encarga de una integración limpia, continua y segura del código de la aplicación.
3. Tras unos minutos, los cambios subidos en la rama (main) deberían de verse reflejados en la aplicación en la nube.

Con github también se usa el CLI de AWS (para la conexión con la instancia), APPSPEC.yml y para definir cómo copiar los archivos del proyecto y que scripts ejecutar según el ciclo de vida de la aplicación (BeforeInstall, AfterInstall, etc).

8 - METODOLOGÍA DE TRABAJO:

El desarrollo de Sushi Miyu se llevó a cabo mediante un enfoque de trabajo colaborativo orientado a simular un entorno profesional real de desarrollo de software.

8.1 ORGANIZACIÓN:

El proyecto fue desarrollado por un equipo de tres integrantes: Ainhoa Quintero González, Joaquín Ruiz Jiménez y Adriana Salazar Daza.

En lugar de construir el proyecto al completo desde el inicio, se fue dividiendo en pequeñas tareas y funcionalidades con el fin de ir desarrollando progresivamente.

8.2 TRELLO Y SPRINTS:

El desarrollo siguió un enfoque iterativo e incremental, dividiéndose en pequeñas funcionalidades que se realizaron mediante Sprints siguiendo una metodología agile, en los que tras el desarrollo de un sprint se revisaba que todo continuara funcionando sin errores.

Los sprints fueron separados por semanas, donde en cada una se avanzaría en varios aspectos a la vez, avanzando en las tablas de Laravel y en el frontend.

Para la organización del proyecto se utilizó Trello, lo cual nos permitió dividir el trabajo y asignarlo de una forma más sencilla. La tabla se dividió en: “product backlog”, “to-do”, “in progress”, “testing” y “done”. Además de reservar dos columnas más para colocar la lista de tareas divididas por sprints.

Es cierto que, a pesar de la organización inicial, debido al comienzo de las prácticas en la dual no se pudo seguir la organización de las tareas tal y como se había planteado y hubo algunos retrasos.

8.3 CONTROL DE VERSIONES:

Para el control de versiones se utilizó Git y GitHub, lo cual nos permite trabajar simultáneamente, evitar pérdida de código, resolver conflictos y mantener un historial.

8.3.1 Estructura de ramas:

En total se utilizaron cuatro ramas a lo largo del desarrollo:

- main: versión estable final para desplegar.
- develop: rama de desarrollo.
- pagInicio: rama creada para desarrollar páginas y funciones en concreto.
- despliegue: creada específicamente para todo el contenido del despliegue.

8.3.2 Convención de commits:

Se adoptó una nomenclatura clara para la convención de commits siguiendo modelos reales de empresas. Para ello se escribe qué cambio se ha agregado utilizando palabras como feat, fix o docs, detrás entre paréntesis que archivos se han cambiado o qué funcionalidad se ha añadido y finalmente se explica más a fondo.

Ejemplos:

- feat(products): crear el product CRUD.
- fix(auth): arreglo el token enviado.
- docs(readme): actualizo el readme con los cambios de React

Más sobre la convención de commits se puede encontrar en el README del proyecto

8.3.3 Release y versionado:

Para la versión entregable final del proyecto se creó un release final etiquetado con una tag en GitHub. Estas versiones y etiquetas se colocan en la rama main.



9 - CONCLUSIONES:

El desarrollo de Sushi Miyu ha permitido diseñar e implementar una solución web completa orientada a cubrir las necesidades reales de un pequeño negocio y poder consolidar los conocimientos adquiridos a lo largo del grado superior.

9.1 OBJETIVOS ALCANZADOS:

Tras el desarrollo realizado, se concluye que los objetivos principales han sido alcanzados satisfactoriamente. Entre algunos de los objetivos logrados se encuentran:

- Visualización de productos y categorías.
- Sistema de autenticación mediante Laravel Sanctum, permitiendo la protección de rutas privadas gracias a la gestión de los tokens y roles.
- Gestión de pedidos, pudiendo cambiar su estado.
- Dashboard administrativo donde gestionar y administrar los datos almacenados.
- Infraestructura cloud utilizando AWS, Docker y Terraform.
- Branding de la marca.

9.2 VALORACIÓN TÉCNICA:

Desde un punto de vista técnico, Sushi Miyu ha supuesto un proyecto de elevada complejidad debido a las múltiples tecnologías y áreas de conocimientos.

Durante el desarrollo se integraron conceptos como:

- Desarrollo de frontend.
- Desarrollo de backend.
- APIs REST.
- Bases de datos relacionales.
- Seguridad web.
- Cloud computing.
- Infraestructura.
- DevOps.
- Diseño UX/UI.

9.3 APRENDIZAJES OBTENIDOS:

Durante el desarrollo hemos podido consolidar conocimientos tanto técnicos como profesionales:

- Angular.
- React.
- Laravel.

- MySQL.
- APIs Rest.
- Docker.
- Terraform.
- AWS.
- Git y GitHub.
- Seguridad y autenticación.

Además de los conocimientos descritos anteriormente se ha mejorado competencias profesionales y habilidades entre las que se destacan:

- Trabajo en equipo.
- Organización del trabajo.
- Adaptación a cambios

9.4 MEJORAS PROPUESTAS:

Durante el planteamiento de los objetivos del proyecto definimos también algunas mejoras que no se pudieron realizar pero que se podrían incorporar en futuras versiones.

- **Sistema de reservas:** permitir a los usuarios reservar mesa, y estas serían posteriormente gestionadas en el dashboard.
- **Alérgenos:** agregar una nueva tabla para almacenar los alérgenos y relacionarla con los productos.
- **Gestión de mesas:** poder almacenar los datos de la cantidad de mesas con las que cuenta el restaurante y saber cual está libre, ocupada si han pedido la cuenta, etc.
- **Cambio de idioma:** implementar un botón junto a su funcionalidad para poder cambiar el idioma de la aplicación de Angular.
- **Envío de tickets:** al realizar el pago correctamente con el pedido se le enviaría un correo automáticamente a la persona con la lista de los productos, fecha del pedido y un mensaje de confirmación.